

1 INTRODUCTION

Tree-based indexes are fundamental to many database systems for accelerating point lookups, range queries, and updates. Among the many B-tree variants surveyed in the literature [1], three are particularly important: the original B-tree, the B⁺-tree, which improves range scans by linking leaves, and the B*-tree, which trades insertion cost for higher node occupancy. In this report, we implement all three from scratch and compare their performance under a range of fan-out parameters across four operation classes (insert, point lookup, range query, and erase) together with structural integrity checks.

Throughout this report we follow the CLRS[2] minimum-degree convention t , where a non-root node holds between $t - 1$ and $2t - 1$ keys. Our implementation exposes the fan-out order d (Knuth's convention, the maximum number of children per node) as the configurable external parameter, and internally converts it to $t = \lceil \frac{d}{2} \rceil$. Because CLRS's parameterization can only express even fan-outs ($d = 2t$), an odd input d rounds up to $d + 1$ (e.g., $d = 5$ yields $t = 3$ with effective max children 6). Throughout the experiments, the reported tree order d corresponds to an internal minimum degree of $t = \lceil d/2 \rceil$.

2 Fundamental Index Structure (B-tree)

2.1 B-Tree

A B-tree is a tree data structure that has the following properties[2]:

- Each node x has $x.n$ keys stored in monotonically increasing order.
- Each internal node x has $x.n + 1$ pointers to its children.
- The i -th key of the node x $x.key_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root i -th child of x , then $k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \dots \leq x.key_{x.n} \leq k_{x.n+1}$
- All leaves have the same depth.
- There is a t for tree which indicates the minimum degree of the tree.
 - Every node other than the root must have at least $t - 1$ keys. Therefore all internal nodes other than the root should have at least t children.
 - Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children.

2.1.1 Search. The *Search* operation finds the key in the node which is greater or equal to the target key and if the key is not equal, goes to the child node and do *Search* recursively. The pseudocode is given in Algorithm 1.

2.1.2 Insert. *Insert* performs the same descent as *Search*, but preemptively invokes *Split-Child* whenever the child it is about to descend into is full ($2t - 1$ keys); this guarantees INSERT-NONFULL never recurses into a node that cannot accommodate one more key. The root, having no parent, is handled separately: a full root is

Algorithm 1 B-Tree Search Algorithm

```

1: procedure SEARCH( $node, key$ )
2:    $i \leftarrow 0$ 
3:   while  $i < |node.keys|$  and  $key > node.keys[i]$  do
4:      $i \leftarrow i + 1$ 
5:   end while
6:   if  $i < |node.keys|$  and  $key = node.keys[i]$  then
7:     return ( $node, i$ )
8:   end if
9:   if  $node.is\_leaf$  then
10:    return NIL
11:  end if
12:  return SEARCH( $node.children[i], key$ )
13: end procedure

```

wrapped in a new internal node and split, which is the only mechanism by which a B-tree grows in height. Algorithm 2 shows the procedure.

2.1.3 Erase. The *Erase* operation finds the key by descending the tree similar to *Search*. The action depends on where the key is found and the cardinality of the relevant child:

- **Key in a leaf node:** simply remove the (key, value) pair from the leaf.
- **Key in an internal node,** left child has $\geq t$ keys: replace the key with its in-order predecessor (the rightmost key in the left subtree), then recursively erase that predecessor from the left subtree.
- **Key in an internal node, right child has $\geq t$ keys:** symmetric to above, using the in-order successor.
- **Key in an internal node, both children at exactly $t - 1$ keys:** merge the two children together with the separator key (forming a single child with $2t - 1$ keys), then recursively erase from the merged child.
- **Key not in the current node:** descend into the appropriate child. Before descent, if that child has $\leq t - 1$ keys, call *Fill* to bring it to $\geq t$ keys (by *Rotate-Right*, *Rotate-Left*, or *Merge-Two* with a sibling). This pre-condition mirrors the preemptive *Split* in *Insert*: the parent ensures the descended child can survive one more deletion without violating the $t - 1$ minimum.

The full procedure is given in Algorithm 3.

Algorithm 2 B-Tree Insert Algorithm

```

1: procedure INSERT( $T, key, value$ )
2:   if  $T.root = NIL$  then
3:      $T.root \leftarrow$  new leaf node with  $(key, value)$ ; return
4:   end if
5:   if  $|T.root.keys| = 2t - 1$  then
6:      $new\_root \leftarrow$  new internal node
7:      $new\_root.children.push\_back(T.root)$ 
8:     SPLIT-CHILD( $new\_root, 0$ )
9:      $T.root \leftarrow new\_root$ 
10:  end if
11:  return INSERT-NONFULL( $T.root, key, value$ )
12: end procedure

13: procedure INSERT-NONFULL( $node, key, value$ )
14:   $i \leftarrow$  index of key in node
15:  if  $i < |node.keys|$  and  $key = node.keys[i]$  then
16:     $node.values[i] \leftarrow value$ ; return
17:  end if
18:  if  $node.is\_leaf$  then
19:    Insert  $(key, value)$  at position  $i$ ; return
20:  end if
21:  if  $|node.children[i].keys| = 2t - 1$  then
22:    SPLIT-CHILD( $node, i$ )
23:    if  $key > node.keys[i]$  then
24:       $i \leftarrow i + 1$ 
25:    end if
26:  end if
27:  return INSERT-NONFULL( $node.children[i], key, value$ )
28: end procedure

29: procedure SPLIT-CHILD( $parent, i$ )
30:   $full\_node \leftarrow parent.children[i]$ 
31:   $sibling \leftarrow$  new node with same  $is\_leaf$  value
32:   $sibling.keys \leftarrow full\_node.keys[t..2t - 2]$ 
33:   $sibling.values \leftarrow full\_node.values[t..2t - 2]$ 
34:  if not  $full\_node.is\_leaf$  then
35:     $sibling.children \leftarrow full\_node.children[t..2t - 1]$ 
36:  end if
37:   $mid\_k \leftarrow full\_node.keys[t - 1]$ 
38:   $mid\_v \leftarrow full\_node.values[t - 1]$ 
39:  truncate  $full\_node$  to its left half
40:  insert  $sibling$  into  $parent.children[i + 1]$ 
41:  insert  $(mid\_k, mid\_v)$  into  $parent$  at  $i$ 
42: end procedure

```

Algorithm 3 B-Tree Erase Algorithm

```

1: procedure ERASE( $T, key$ )
2:   if  $T.root = NIL$  then
3:     return
4:   end if
5:   DO-ERASE( $T.root, key$ )
6:   if  $|T.root.keys| = 0$  then
7:      $old \leftarrow T.root$ 
8:     if  $old.is\_leaf$  then
9:        $T.root \leftarrow NIL$ 
10:    else
11:       $T.root \leftarrow old.children[0]$ 
12:    end if
13:    free  $old$ 
14:  end if
15: end procedure

16: procedure DO-ERASE( $node, key$ )
17:   $i \leftarrow$  index of key in node
18:  if  $i < |node.keys|$  and  $key = node.keys[i]$  then
19:    if  $node.is\_leaf$  then
20:      remove  $(node.keys[i], node.values[i])$ 
21:    else
22:      ERASE-INTERNAL( $node, i$ )
23:    end if
24:    return
25:  end if
26:  if  $node.is\_leaf$  then
27:    return
28:  end if
29:   $last \leftarrow (i = |node.keys|)$ 
30:  if  $|node.children[i].keys| \leq t - 1$  then
31:    FILL( $node, i$ )
32:  end if
33:  if  $last$  and  $i > |node.keys|$  then
34:    DO-ERASE( $node.children[i - 1], key$ ); return
35:  end if
36:  DO-ERASE( $node.children[i], key$ )
37: end procedure

38: procedure ERASE-INTERNAL( $node, i$ )
39:   $L \leftarrow node.children[i]$ ;  $R \leftarrow node.children[i + 1]$ 
40:  if  $|L.keys| \geq t$  then
41:     $(k, v) \leftarrow$  rightmost  $(key, value)$  in  $L$ 
42:     $node.keys[i] \leftarrow k$ ;  $node.values[i] \leftarrow v$ 
43:    DO-ERASE( $L, k$ )
44:  else if  $|R.keys| \geq t$  then
45:     $(k, v) \leftarrow$  leftmost  $(key, value)$  in  $R$ 
46:     $node.keys[i] \leftarrow k$ ;  $node.values[i] \leftarrow v$ 
47:    DO-ERASE( $R, k$ )
48:  else
49:     $key \leftarrow node.keys[i]$ 
50:    MERGE-TWO( $node, i$ )
51:    DO-ERASE( $node.children[i], key$ )
52:  end if
53: end procedure

54: procedure FILL( $parent, i$ )
55:  if  $i > 0$  and  $|parent.children[i - 1].keys| \geq t$  then
56:    ROTATE-RIGHT( $parent, i$ )
57:  else if  $i < |parent.keys|$  and  $|parent.children[i + 1].keys| \geq t$  then
58:    ROTATE-LEFT( $parent, i$ )
59:  else

```

2.1.4 Range Query. The *Range* operation performs an in-order traversal bounded by $[low, high]$, descending into a child's subtree before visiting each key whose value falls in the range. Algorithm 4 gives the pseudocode.

3 Advanced Index Structure (B+-tree, B*-tree)

3.1 B⁺-Tree

A B⁺-tree shares the structural invariants of a B-tree (Section 2.1) but differs in two key ways: (1) all satellite data (record pointers) is stored only in leaf nodes, while internal nodes hold only routing keys; and (2) leaf nodes are linked as a doubly-linked list to support efficient sequential access. Internal-node routing uses strict inequalities — if k is in the subtree of the i -th child then $k < x.key_i$, and equal keys descend to the right.

3.1.1 Search. Since B⁺-tree contains data only in leaf nodes, *Search* first descends to the leaf containing the key, then searches within the leaf. The pseudocode is given in Algorithm 5.

Algorithm 4 B-Tree Range Query

```

1: procedure RANGE(node, low, high, visit)
2:    $i \leftarrow 0$ 
3:   while  $i < |node.keys|$  and  $node.keys[i] < low$  do
4:      $i \leftarrow i + 1$ 
5:   end while
6:   while  $i < |node.keys|$  and  $node.keys[i] \leq high$  do
7:     if not node.is_leaf then
8:       RANGE(node.children[ $i$ ], low, high, visit)
9:     end if
10:    VISIT(node.keys[ $i$ ], node.values[ $i$ ])
11:     $i \leftarrow i + 1$ 
12:   end while
13:   if not node.is_leaf then
14:     RANGE(node.children[ $i$ ], low, high, visit)
15:   end if
16: end procedure

```

Algorithm 5 B⁺ Tree Search Algorithm

```

1: procedure SEARCH(T, key)
2:   leaf  $\leftarrow$  FIND-LEAF(T.root, key)
3:    $i \leftarrow$  first index such that  $leaf.keys[i] \geq key$ 
4:   if  $i < |leaf.keys|$  and  $leaf.keys[i] = key$  then
5:     return (leaf,  $i$ )
6:   end if
7:   return NIL
8: end procedure
9: procedure FIND-LEAF(node, key)
10:  while not node.is_leaf do
11:     $i \leftarrow$  first index such that  $key < node.keys[i]$ 
12:    node  $\leftarrow$  node.children[ $i$ ]
13:  end while
14:  return node
15: end procedure

```

Algorithm 6 B⁺ Tree Insert Algorithm

```

1: procedure SPLIT-LEAF-CHILD(parent,  $i$ )
2:   full_node  $\leftarrow$  parent.children[ $i$ ]
3:   sibling  $\leftarrow$  new leaf node
4:   sibling.keys  $\leftarrow$  full_node.keys[ $t \dots 2t - 1$ ]
5:   sibling.values  $\leftarrow$  full_node.values[ $t \dots 2t - 1$ ]
6:   truncate full_node to first  $t$  keys
7:   LINK-LEAF-LIST(full_node, sibling)
8:   router  $\leftarrow$  sibling.keys[0]
9:   insert sibling into parent.children at position  $i + 1$ 
10:  insert router into parent.keys at position  $i$ 
11: end procedure
12: procedure SPLIT-INTERNAL-CHILD(parent,  $i$ )
13:    $\triangleright$  Same logic with Split-Child in B-Tree
14: end procedure

```

Algorithm 7 B⁺ Tree Erase Algorithm

```

1: procedure DO-ERASE(node, key)
2:   if node.is_leaf then
3:     if key is found in node then
4:       remove (key, value)
5:     end if
6:     return
7:   end if
8:    $i \leftarrow$  first index such that  $key < node.keys[i]$ 
9:   if  $|node.children[i].keys| \leq t - 1$  then
10:     $i \leftarrow$  FILL(node,  $i$ )
11:   end if
12:   DO-ERASE(node.children[ $i$ ], key)
13: end procedure

```

3.1.2 Insert. The *Insert* operation of the B⁺-tree follows the same descent as B-tree (Algorithm 2). The leaf split, however, is slightly different: instead of moving the median key up to the parent, the median is *copied* to the parent as a routing key while remaining in the leaf. The internal split is identical to B-tree's. The leaf split procedure is shown in Algorithm 6.

3.1.3 Erase. The *Erase* method follows the same logic as B-tree, but with two simplifications: actual key removal only occurs at leaf nodes, and the case-analysis of B-tree's ERASE-INTERNAL is unnecessary because internal keys are mere routing copies. Stale internal routing keys are tolerated since they still correctly partition the subtrees. MERGE-TWO is identical to B-tree's, except that when merging two leaves it additionally updates the leaf-list pointer ($L.next \leftarrow R.next$). Algorithm 7 shows the modified DO-ERASE.

3.1.4 Range. Since the B⁺-tree links its leaf nodes and stores actual data only in leaves, *Range* leverages this structure: it finds the leaf containing the low key and iteratively traverses the leaf list, instead of performing an in-order recursion as in B-tree. The pseudocode is given in Algorithm 8.

Algorithm 8 B+ Tree Range Query

```

1: procedure RANGE( $T, low, high, visit$ )
2:    $leaf \leftarrow \text{FIND-LEAF}(T.root, low)$ 
3:    $i \leftarrow$  first index such that  $leaf.keys[i] \geq low$ 
4:   while  $leaf \neq \text{NIL}$  do
5:     for  $j \leftarrow i$  to  $|leaf.keys| - 1$  do
6:       if  $leaf.keys[j] > high$  then
7:         return
8:       end if
9:       VISIT( $leaf.keys[j], leaf.values[j]$ )
10:    end for
11:     $leaf \leftarrow leaf.next$ 
12:     $i \leftarrow 0$ 
13:  end while
14: end procedure

```

3.2 B*-Tree

A B*-tree is a variant of the B-tree introduced by Knuth[4] that aims for higher average node occupancy. While a standard B-tree guarantees $\geq t - 1$ keys per non-root node ($\approx 1/2$ full), a B*-tree targets approximately $2/3$ occupancy by deferring splits.

3.2.1 Insert Policy. A B*-tree shares the same structural invariants as a B-tree (same capacity, leaf-depth, routing semantics); only the split policy on insertion differs. When a child becomes full, the algorithm first attempts **cooperative redistribution**: if an adjacent sibling has spare capacity, keys are redistributed between them and the parent's separator is updated, allocating no new node. Only when both adjacent siblings are also full does the algorithm fall back to a $2 \rightarrow 3$ **split**: the full child and one full sibling (a combined $4t - 3$ keys including the separator) are repartitioned into three nodes of approximately $\lceil (4t - 3)/3 \rceil$ keys each, promoting two new separators. Newly created nodes thus start at $\approx 2/3$ capacity instead of B-tree's $1/2$. Algorithm 9 shows the modified DO-SPLIT; REDISTRIBUTE-WITH-LEFT is the symmetric mirror of REDISTRIBUTE-WITH-RIGHT and is omitted.

3.2.2 Erase Policy. Maintaining a strict $2/3$ minimum on erase would require a $3 \rightarrow 2$ merge mechanism symmetric to the insert-side $2 \rightarrow 3$ split. While such a primitive can be defined, it offers no asymptotic benefit: erase already runs in $O(t \log_t n)$, and the strict- $2/3$ variant only shifts the post-deletion lower bound on occupancy from $\approx 1/2$ to $\approx 2/3$. Furthermore, the strict variant introduces corner cases (e.g., a non-leaf root with two children both at the strict minimum, whose combined size exceeds capacity) that require additional rebalancing logic. Since the higher fill factor of a B*-tree is primarily established by insertions and persists as long as deletions are infrequent, we adopt B-tree's simpler erase semantics: *Erase*, *Erase-Internal*, and *Fill* are inherited unchanged from B-tree (Algorithm 3). The $\approx 2/3$ occupancy is thus a statistical tendency from insertions, not an invariant maintained on every deletion.

3.3 Time Complexity Comparison

For all three trees, the height is $O(\log_t n)$ and per-node key search is $O(t)$ (linear) or $O(\log t)$ (binary), giving Search/Insert/Erase costs of $O(t \log_t n)$ or $O(\log n)$ respectively. The B*-tree's Search always

Algorithm 9 B* Tree Split and Redistribution

```

1: procedure DO-SPLIT( $parent, i$ )
2:    $M \leftarrow 2t - 1$ 
3:   if  $i > 0$  and  $|parent.children[i - 1].keys| \leq M - 2$  then
4:     REDISTRIBUTE-WITH-LEFT( $parent, i$ )
5:     return
6:   else if  $i < |parent.keys|$  and  $|parent.children[i + 1].keys| \leq M - 2$  then
7:     REDISTRIBUTE-WITH-RIGHT( $parent, i$ )
8:     return
9:   end if
10:  if  $i < |parent.keys|$  then
11:    SPLIT-TWO-INTO-THREE( $parent, i$ )
12:  else if  $i > 0$  then
13:    SPLIT-TWO-INTO-THREE( $parent, i - 1$ )
14:  else
15:    SPLIT-IN-TWO( $parent, i$ )
16:  end if
17: end procedure
18: procedure REDISTRIBUTE-WITH-RIGHT( $parent, i$ )
19:    $L \leftarrow parent.children[i]$ 
20:    $R \leftarrow parent.children[i + 1]$ 
21:    $K \leftarrow L.keys ++ [parent.keys[i]] ++ R.keys$ 
22:    $V \leftarrow$  analogous for values
23:    $new\_L \leftarrow \lfloor |K|/2 \rfloor$ 
24:    $L.keys, L.values \leftarrow K[0 \dots new\_L - 1]$ 
25:    $parent.keys[i], parent.values[i] \leftarrow K[new\_L]$ 
26:    $R.keys, R.values \leftarrow K[new\_L + 1 \dots ]$ 
27:    $\triangleright$  Children pointers redistributed analogously when not
      leaves
28: end procedure

```

Algorithm 9 B* Tree Split and Redistribution (Continued)

```

1: procedure SPLIT-TWO-INTO-THREE( $parent, i$ )
2:    $L \leftarrow parent.children[i]$ 
3:    $R \leftarrow parent.children[i + 1]$ 
4:    $K \leftarrow L.keys ++ [parent.keys[i]] ++ R.keys$ 
5:    $V \leftarrow$  analogous for values
6:    $keys\_to\_distribute \leftarrow |K| - 2$ 
7:    $a \leftarrow \lceil keys\_to\_distribute/3 \rceil$ 
8:    $b \leftarrow \lceil (keys\_to\_distribute - a)/2 \rceil$ 
9:    $M\_node \leftarrow$  new node
10:   $L.keys \leftarrow K[0 \dots a - 1]$ 
11:   $sep1 \leftarrow K[a]$ 
12:   $M\_node.keys \leftarrow K[a + 1 \dots a + b]$ 
13:   $sep2 \leftarrow K[a + b + 1]$ 
14:   $R.keys \leftarrow K[a + b + 2 \dots ]$ 
15:   $parent.keys[i] \leftarrow sep1$ 
16:  insert  $sep2$  into  $parent.keys$  at position  $i + 1$ 
17:  insert  $M\_node$  into  $parent.children$  at position  $i + 1$ 
18: end procedure

```

descends to a leaf even when an internal routing key matches, which adds only a constant factor. Range queries cost $O(\log_t n + k)$ asymptotically for all three; however, the B⁺-tree leaf-list scan touches $\lceil k/B \rceil$ contiguous nodes (where B is leaf occupancy), whereas the B-tree and B^{*}-tree in-order recursion additionally visits $O(k/t)$ internal nodes — a constant-factor advantage for B⁺-tree on cache- or disk-bound workloads. The B^{*}-tree Insert keeps the same asymptotic bound, but each split on the path performs cooperative redistribution or $2 \rightarrow 3$ split at a $\Theta(t)$ cost — roughly a 2–3× constant-factor overhead compared to B-tree’s simple $1 \rightarrow 2$ split, traded for higher fill factor.

4 Experimental Results and Analysis

4.1 Experiment Setup

We insert 100,000 records from the provided CSV file into each tree, then perform 100,000 point lookups (a mix of inserted keys and random keys that may not be present), several range queries of varying sizes computing the sum of $1000 \times \text{GPA}$ for all students within the range, and finally 50,000 random key deletions. During the deletion phase, a structural check verifies that the tree satisfies its invariants (per-node key counts, leaf-depth uniformity, and leaf-list connectivity for B⁺-tree). Each timing point is the average of 10 repetitions, with two warmup runs to mitigate cache and frequency effects. The environment is WSL2 on Windows 11 with an Intel Core Ultra 7 258V CPU and 16 GB RAM. The performance code is compiled with `-std=c++17 -O2`; structural integrity was independently verified under a separate `-fsanitize=address` build, which reported zero violations across all workloads. The raw data and implementation code are available at <https://github.com/hikari0102/SimpleIndexingImpl>.

4.2 Insertion and Parameter Tuning

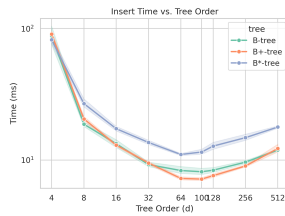


Figure 1: Performance graph for inserting 100,000 records

Table 1: Structural events during random insertion of 100,000 keys (selected d). $r = \text{redists} / (\text{splits} + \text{redists})$ for B⁺-tree.

d	B-tree	B ⁺ -tree	B [*] -tree splits	B [*] -tree redists	r
4	57,102	82,352	51,272	25,314	33%
16	9,803	10,473	8,072	17,100	68%
64	2,295	2,360	1,846	10,433	85%
256	536	541	446	5,117	92%

As shown in Figure 1, B⁺-tree exhibits the worst performance across all orders while B-tree and B^{*}-tree are nearly indistinguishable, and

the performance forms a U-shape with a sweet spot at $d \approx 64$ –100. The B^{*}-tree penalty stems from its split policy: rather than performing a simple $1 \rightarrow 2$ split, it first attempts cooperative redistribution (rewriting two adjacent nodes), and falls back to a $2 \rightarrow 3$ split (rewriting three nodes plus the parent) when both siblings are full — both at $\Theta(d)$ memory-copy cost per occurrence. The U-shape itself reflects the well-known trade-off between tree height ($O(\log_d n)$, favoring large d) and intra-node cost [3]: with each cache line capable of holding only a fixed number of keys, increasing d beyond the sweet spot ($d \approx 100$) brings diminishing returns on height while linearly increasing the per-node memory shift cost.

Table 1 confirms this quantitatively: B^{*}-tree’s redistribute share rises monotonically from 33% at $d = 4$ to over 92% at $d \geq 256$, since larger nodes are more likely to have spare sibling capacity. Although B^{*}-tree allocates fewer nodes overall, each redistribute still rewrites two adjacent nodes at $\Theta(d)$ cost, and these events are several times more frequent than the splits they avoid — explaining its persistent insertion penalty. The B⁺-tree’s higher split count at small d reflects leaf-level splits frequently propagating to internal-level splits when leaves are tiny; at $d \geq 64$ the two trees converge.

4.3 Indexing and Parameter Tuning

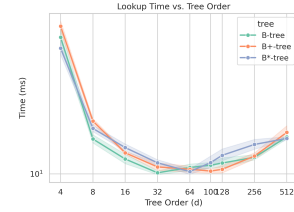


Figure 2: Performance graph for searching 100,000 records

All three trees show nearly identical lookup performance for $d \geq 8$, with a U-shape and sweet spot around $d \approx 64$ where tree height is minimized while intra-node search remains cheap. **At $d = 4$, however, the B^{*}-tree is fastest while B⁺-tree is slowest.** This inversion at small d reflects two compensating effects: B^{*}-tree’s higher fill factor (≈ 0.78 vs ≈ 0.58) yields a tree one level shallower, saving one full memory hop per lookup; B⁺-tree, conversely, must always descend to a leaf even when an internal routing key matches, paying the extra hop unconditionally.

For $d \geq 100$, B^{*}-tree gradually becomes slower than the other two: its denser nodes incur more intra-node comparisons per descent — the same density that helped at small d becomes a liability when height is no longer the dominant cost. The U-shape across d reflects the same height vs intra-node trade-off as insertion.

4.4 Range query and Tree types

Note that the y-axis of Figure 3 is log-scaled. B⁺-tree consistently performs the best across all d values, while B-tree and B^{*}-tree are similar; this is because B⁺-tree exploits its leaf-list links to scan range results sequentially without descending back through internal nodes. The U-shape disappears here because range queries are read-only and bypass the $\Theta(d)$ memory shifts that penalize insert and erase at large d .



Figure 3: Performance graph for range query

4.5 Erase and Integrity

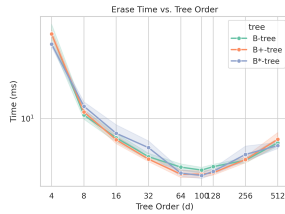


Figure 4: Performance graph for erase

As shown in fig. 4, the deletion performance forms a U-shaped curve with a sweet spot around $d \approx 64-128$, after which the $\Theta(d)$ memory shift cost within each leaf dominates. The relative ordering reveals two effects.

B⁺-tree is consistently the fastest across the middle range ($d = 8$ to 256). Since its data resides only in leaves, every erase reduces to a leaf-level removal followed by underflow handling, and the case-analysis of *Erase-Internal* (descending to a predecessor or successor in the opposite subtree) is never invoked. **B⁺-tree is consistently the slowest across the same range**, even though it inherits B-tree's erase semantics. The cause is its higher fill factor itself: deleting a key from a leaf shifts all subsequent keys by one position, and a denser leaf simply has more keys to shift. At the extremes ($d = 4$ and $d = 512$) the differences vanish as either tree-traversal or per-leaf shift cost dominates uniformly across all three trees.

For integrity, we periodically invoked a structural check (every 5,000 deletions, 10% of the workload) verifying ordering, key/child counts, leaf-depth uniformity, and leaf-list pointers in B⁺-tree, plus a final cross-check against `std::map`; both reported zero violations across all configurations.

5 Additional Experiments: Insert order, Fill factor, and Height

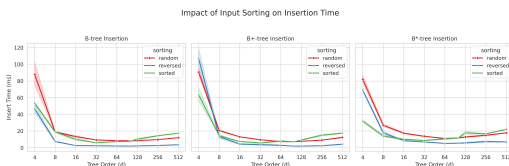


Figure 5: Performance graph for various input order

For B-tree and B⁺-tree, **reversed input is consistently the fastest, while sorted is comparable to random and becomes the slowest at large d ($d \geq 256$)**. Although sequential inputs (sorted/reversed) confine activity to a single root-to-leaf path, the asymmetry traces to our split implementation: split retains the left half in the original node and allocates a new *right sibling*. Reversed input keeps the leftmost path active, reusing the same memory and benefiting from cache locality, whereas sorted input continually shifts work into freshly allocated right siblings. **For B⁺-tree the pattern reverses: sorted is fastest**. Under random input B⁺-tree frequently triggers cooperative redistribution at $\Theta(d)$ cost; under sequential input the active leaf has no sibling on the active side, so redistribution never fires and B⁺-tree falls back to a plain split.

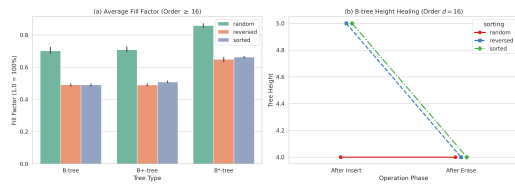


Figure 6: Fill factor and height difference

To deeply understand the variants' internal behavior, we extracted the average node fill factor and tree height across workloads (Figure 6). Under random insertion, B-tree and B⁺-tree converge to ≈ 0.70 , close to Yao's predicted $\ln 2 \approx 0.69$, while B⁺-tree reaches ≈ 0.85 , exceeding Knuth's $2/3$ target due to frequent redistributions. Under sequential input, fill drops to ≈ 0.50 for B-tree/B⁺-tree and ≈ 0.65 for B⁺-tree — consistent with the redistribution-bypass effect noted above: sequential growth always splits the active leaf exactly in half, producing the structural minimum. The lower occupancy inflates the height by one level (Figure 6(b)). After 50% random deletion, however, the underflow-driven merges consolidate the under-occupied leaves, healing the tree back to the same height as a randomly built one — evidence that the sequential-input penalty is structural, not fundamental.

6 Conclusion

B⁺-tree's higher fill factor is a double-edged sword: it pays off only at very small d where one less level of height accelerates lookups, but at every other order it makes B⁺-tree the slowest for both insertions and deletions due to denser intra-node memory shifts. The B⁺-tree's leaf-list scan provides a constant-factor advantage on range queries that is insensitive to per-node density. Sequential bulk loads silently disable B⁺-tree's redistribution mechanism, so its occupancy advantage vanishes for bulk-built indexes. For our 100,000-record dataset the practical sweet spot is $d \approx 64-128$, beyond which the $\Theta(d)$ intra-node cost dominates the logarithmic gain in tree height.

References

- [1] Douglas Comer. 1979. Ubiquitous B-Tree. *Comput. Surveys* 11, 2 (1979), 121–137. doi:10.1145/356770.356776
- [2] Thomas H. Cormen, Charles Eric Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. The MIT Press.
- [3] Goetz Graefe and Per-Åke Larson. 2001. B-Tree Indexes and CPU Caches. In *Proceedings of the 17th International Conference on Data Engineering (ICDE)*. IEEE, 349–358. doi:10.1109/ICDE.2001.914847

- [4] Donald E. Knuth. 1998. *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.